# Detection of Software Update Path Risks in Open Text Stream

Anonymous Authors

***Abstract*—Fast maintenance cycles and early detection of software issues are critical in modern software ecosystems. Each update path, from any previous version to a newer version, can introduce functional, security, or compatibility risks. Testing path variants in real-world conditions is difficult, and user feedback on update failures or regressions is often hidden within large volumes of open text data. Parsing such texts for early software update risks is important yet time-consuming. We present a real-time open text pipeline that uses supervised machine learning to detect and classify update-related discussions across online open platforms. The system filters relevant update risks for downstream analysis and supports early identification of issues along the software update path.**

***Index Terms*—component, formatting, style, styling, insert**

## I. Introduction

Performing software updates is inherently a risk that is difficult to fully mitigate during testing. Given the complexity of modern software ecosystems, testing all component variants, configurations, and dependency chains before release is nearly impossible. As a result, many errors and regressions are only discovered after deployment, when updates are applied by end users. Open text platforms such as Reddit or app store feedback provide a valuable channel for users to share update-related risks and system failures. This is particularly important in domains such as the Internet of Things (IoT) or Cyber-Physical Systems (CPS), where operational safety often depends on the reliability of software updates distributed at scale [1], [2].

Fast maintenance cycles and early detection of software issues are therefore critical in modern software ecosystems. Each update path, from any previous version to a newer version, can introduce functional, security, or compatibility risks. Regression testing these paths is a time-consuming and difficult task, especially when software systems depend on faster update cycles as well as complex dependencies and configurations. User feedback about update failures or regressions often appears first in open online discussions but remains hidden within large volumes of unrelated content. Parsing such open text for early software update risks is important, yet unrealistic to perform manually due to the high data volume (see Figure 1).

To address this challenge, we present a real-time open-text pipeline that uses supervised machine learning to detect and classify update-related risks across open online platforms. For this work, we use Reddit as the primary data source for training, testing, and prediction. The dataset consists of 85,000 Reddit software-related posts integrated into an existing data lake of more than 20,000 software components and 120,000 software component versions. Beyond Reddit, data sources include MITRE CVE, GitHub, Wikipedia, DistroWatch, and many others.

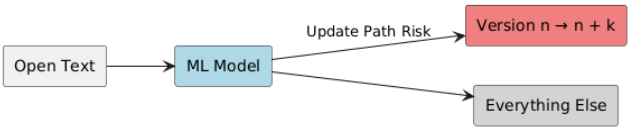



Fig. 1. Filtering software update risks using a machine learning model. The model identifies risk-related version paths while separating non-critical updates.

Reddit posts span core system components across operating systems, browsers, and data services. They highlight how failures in software updates can cascade across systems, affecting both system reliability and user experience.

Recent sample posts include:

- r/chrome: *"OpenGL is broken since latest update 139.0.7258.67"* [3]
- r/Edge: *"Edge hangs and crashes after update from version 140.0.3485.94 to 141.0.3537.57"* [4]
- r/Dell: *"Black screen after Windows update"* [5]
- r/TrueNAS: *"GPT header corruption after upgrading to TrueNAS Scale 25.10"* [6]
- r/Immich: *"DB not starting after V2.0.0 upgrade"* [7]

This detection builds on previous work showing that monitoring and integrating software update data alone is not sufficient for fast risk detection. Earlier systems already collected structured data, such as release notes from developers, CVE records for security risks, and Reddit discussions for user feedback. However, the main bottleneck after a software release remains the lack of automation in identifying emerging risks within large volumes of unstructured user discussions.

For example, in April 2026, the National Institute of Standards and Technology (NIST) announced that the National Vulnerability Database (NVD) would prioritize analysis efforts due to the growing volume of reported CVEs. As a result, newly disclosed vulnerabilities may remain publicly visible through the CVE List while awaiting formal NVD characterization and enrichment [8]. This increasing volume creates a prioritization challenge for both vulnerability analysts and organizations that must make update and mitigation decisions with limited official guidance. To help address this challenge, this work automatically identifies software update path risks from open-text sources, filtering large volumes of community-generated content to surface potential high priority security,

TABLE I
SUMMARY OF RELATED STUDIES USING REDDIT AND SOCIAL PLATFORM DATA

| Author | Data Source(s) | Objective |
|---|---|---|
| Mockus (2000) | Software repositories | Predict software change risk. |
| Fiallos and Jimenes (2019) | Reddit, Twitter | Classify user interests. |
| Shrestha et al. (2020) | GitHub, Twitter, Reddit | Detect vulnerability signals. |
| Iqbal et al. (2021) | Reddit | Extract software requirements. |
| Long et al. (2024) | Reddit | Compare deep text classification models. |
| Singh (2024) | Reddit, release notes | Link posts to software update types. |
| This paper | Reddit | Detect risks in the software update path. |

compatibility, and reliability update issues.

Overall, this study contributes in two main aspects: *Machine Learning* and *Software Systems*.

**Machine Learning:** We develop and evaluate supervised classifiers for detecting update-related risks within unstructured text. The models are trained and tested on labeled Reddit data and evaluated across seven quality dimensions: timeliness, accuracy, variety, completeness, consistency, validity, and uniqueness. The ML models automate risk identification within high-volume discussions and is integrated directly into the system pipeline for real-time classification and continuous feedback.

**Software Systems:** We implement an end-to-end integration pipeline that continuously collects, filters, and stores open text data from multiple platforms. The system includes real-time ingestion, normalization, and enrichment components to support scalable monitoring of software update risks. It integrates the ML classifier for automated prediction and provides downstream interfaces for analytics related to software evolution, vulnerability tracking, and release coordination. A frontend dashboard enriches software updates with detected Reddit software update risks by category, component type, or chronological order.

Unlike prior work that focuses on vulnerability disclosure awareness or software requirements mining, this work focuses on automatically prioritizing software update path risks from large-scale open-text streams to support maintenance and vulnerability assessment workflows.

Towards this objective, Section II reviews the related literature, Section III outlines the methodology, and Section IV presents the results. Section V discusses their implications for data quality and maintenance workflows, followed by Section VI which outlines the limitations and future work. Finally, Section VII concludes the paper.

## II. RELATED WORK

Open text platforms such as Reddit and Twitter have increasingly been used in software engineering and data mining research to extract actionable insights from informal community discussions. Table I summarizes representative studies that use social or repository-based data for tasks such as user interest classification, vulnerability signal detection, requirements extraction, and software change risk prediction. These studies demonstrate the value of community-generated text for identifying relevant events, classifying content, and supporting software maintenance workflows. Our work builds on this direction by focusing specifically on detecting software update-related risks from Reddit posts.

Fiallos and Jimenes used Reddit data as an auxiliary training source to classify Twitter users' interests through multi-label text classification [9]. Their study aimed to bridge communities across platforms and model user preferences using shared linguistic patterns. Similar to our work, they used Reddit as a high-volume informal text source that requires preprocessing and text representation techniques such as Word2Vec and TF–IDF. However, their focus was user interest profiling, while our study focuses on identifying update-related posts that may indicate software risks.

Shrestha et al. examined how GitHub, Twitter, and Reddit can provide early signals of software vulnerabilities [10]. Their work measured public awareness and communication of vulnerability disclosures across multiple social ecosystems. This is related to our use of open text streams for extracting actionable software engineering insights. However, their study focuses on vulnerability awareness and community response, whereas our work targets software risk detection along the update path as a specific event type relevant to maintenance and lifecycle management.

Mockus introduced an early quantitative approach for estimating software change risk using historical modification data from large-scale software repositories [11]. The study showed that predictive modeling can help identify components with a higher likelihood of defects after software changes. While this work relies on structured repository data and code-level metrics, our study extends the idea of software risk detection to unstructured user feedback from Reddit.

Iqbal et al. explored Reddit as a source for software requirements elicitation and used machine learning methods, including support vector machines, to identify software features discussed by users [12]. Their work shows that social media discussions can reveal useful software engineering information. However, their goal was feature extraction and requirements mapping, while our objective is update detection and validation, with an emphasis on timeliness and data quality in open text streams.

Long et al. compared deep learning models, including TextCNN and TextRNN, for classifying Reddit text [13]. Their study focused on evaluating model performance for social media text classification. Like our work, they addressed noisy and variable Reddit content using machine learning methods. In contrast, our study applies machine learning to an update-related issue detection pipeline that emphasizes real-time filtering and data quality improvement.

Singh used Reddit discussions to enrich software release

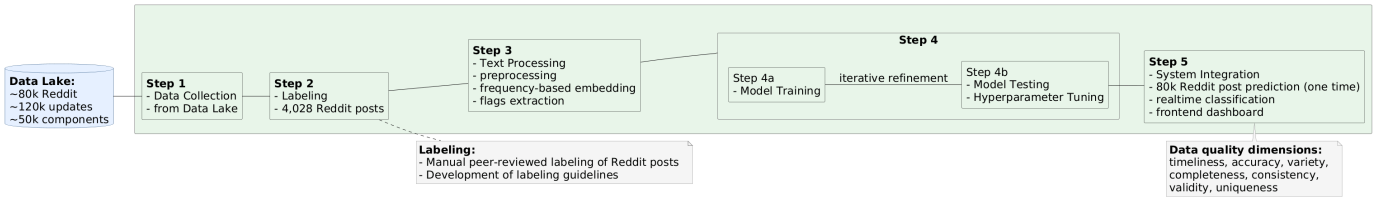



Fig. 2. Real-time streaming pipeline for update-related risk filtering. (a) shows data ingestion and preprocessing; (b) depicts classification, active learning, and feedback integration.

notes and prioritize updates within a clinical software ecosystem [14]. While that work improves documentation after release, our study detects update-related risks directly from unstructured Reddit posts. This distinction allows our approach to support earlier identification of potential software update issues.

Overall, prior studies show the growing importance of social and community-based data in software engineering research. Our work builds on this foundation by narrowing the focus to software update path risk detection and introducing a targeted pipeline for improving reliability and temporal awareness in open text streams.

## III. Method

### A. Data Collection (Step 1)

The pipeline with an extract load transform (ELT) process of raw text streams and batch data from diverse sources, including GitHub repositories, Wikipedia articles, DistroLinux logs, the MITRE CVE database (approximately 100,000 entries), and Reddit posts related to software components (approximately 80,000 entries), as well as issue trackers. All collected data is consolidated into a unified data lake for subsequent preprocessing and analysis. At this stage, metadata such as timestamps, author identifiers, and platform tags are preserved to support later enrichment and validation. Data is extracted in batch every four hours.

### B. Data Labeling (Step 2)

A total of 4,028 Reddit posts were manually annotated as *update risk-related* (True) or *not update-related* (False). Update-related refers to posts where the user describes a technical issue that occurred after a software update. General discussions about update announcements, technical problems that appeared before an update, or troubleshooting unrelated to an update were labeled as False.

To ensure consistency and reduce subjective labeling bias, we followed a structured decision guideline (see Figure 3) which specifies how to determine whether a post describes a troubleshooting issue from a software update. All labels were independently reviewed by a second team member, and disagreements were resolved through discussion.

### C. Text Processing (Stages 3)

Raw textual data in Reddit posts undergoes NLP preprocessing steps, including lowercasing, cleaning, tokenization, and normalization to ensure consistency throughout the data

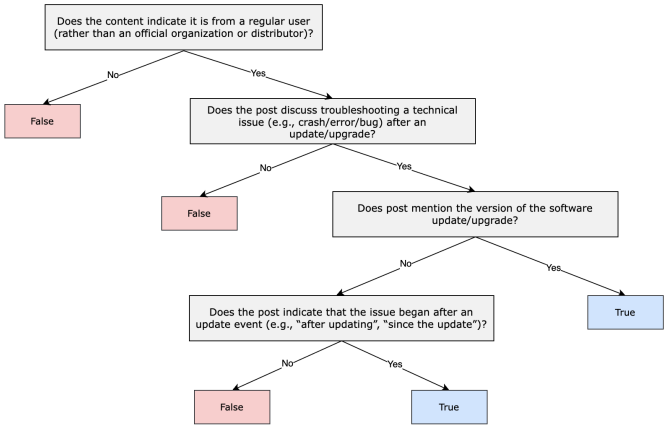



Fig. 3. Decision guideline for manual labeling of update-related posts.

and increase text purity for further processing. During this process, CVE identifiers and software versions were parsed with regular expressions and either or standardized into categories or removed. Software versions were extracted with regular expressions and standardized into four categories: major, minor, patch, and build. For example, “Chrome 140.0.7339.249” is normalized to “chrome version patch”. While, ”CVE-2025-9867” is simplified to ”cve” to help the model better generalize update-related patterns.

Cleaned text is processed using the spaCy library to produce lemmatized tokens and remove stopwords. After these preprocessing steps, the post title and author description fields are concatenated into a single combined text representation to support feature extraction, model training, and testing.

In feature engineering, two groups of features were constructed. The first group consists of five boolean flags using regular expressions to identify specific keywords and patterns in the combined text. The second group consists of TF–IDF (Term Frequency–Inverse Document Frequency) feature sets generated by vectorizing the combined text using hyperparameters, such as n-grams. On the TF-IDF feature set, we did feature selection using the chi-square testing function to select the top 1,700 features most strongly correlate with the target classification label.

### D. Modeling Training & Testing (Step 4)

This stage integrates three supervised classifiers to detect update-related posts, including Logistic Regression, Support

Vector Machine (SVM), and Random Forest. The dataset was divided into training and testing subsets using a 70/30 stratified split to preserve the original class distribution. Due to the presence of imbalance in the dataset (see Figure 4), resampling methods such as random oversampling are applied to the training subset only, to prevent the model from bias toward the majority class.

Model performance was evaluated using Accuracy, Precision, Recall, F1 score, ROC-AUC, and PR-AUC to capture how well it performs overall and on the minority class. Additionally, all three classifiers were tested with different combinations of hyperparameters using the GridSearchCV library to identify ones that result in the best overall metrics.

### E. System Integration (Step 5)

The final step of the method connects the machine learning classifiers with the production data lake and dashboard system. This stage integrates the predicted output by providing structured API endpoints, extending data schemas, and frontend interfaces for visualization and filtering.

*a) Label management:* Dedicated API endpoints allow creating new labels, retrieving existing ones, and filtering posts by positive or negative values. This enables both human annotators and automated classifiers to contribute to the same labeled dataset, ensuring consistency and continuous improvement of model accuracy over time.

*b) Paging and limits:* All endpoints implement `limit` and `page`/`offset` parameters to efficiently handle large datasets and maintain low query latency. For example, `label=positive&limit=500&page=3` retrieves a deterministic subset of results suitable for batch prediction, data validation, or dashboard rendering. Before running full-scale predictions, a small-sample test can be performed using the same parameters (e.g., the first 200–500 posts) to validate schema consistency, output ranges, and model score values. This incremental approach reduces operational risk by identifying potential data or configuration errors early.

*c) Extended schema design:* The system provides a unified data schema that goes beyond the machine learning output to meet frontend and user requirements. Each record includes an `id`, `text`, `component`, `version`, `label`, `score`, and `timestamp`, along with structured namespaces such as `metadata.*`. This structure ensures that prediction outputs remain compatible with user-facing systems and preserve traceability for audits and retraining.

*d) Integration and visualization:* The filtered update-related posts are stored in the dashboard database and rendered through an interactive web interface. Users can filter results by component, risk score, or chronological order to inspect potential risks along the software update path. Each entry links back to the original Reddit post and displays its associated metadata and model confidence, providing a complete view of emerging risks in near real time.

Together, these integration mechanisms ensure that the classifier's predictions transition smoothly from experimental evaluation to production deployment, supporting real-time monitoring, reproducibility, and interactive analysis within the larger update-tracking ecosystem.

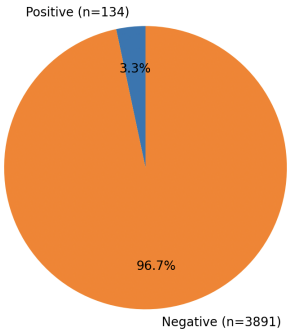



Fig. 4. Dataset class distribution.

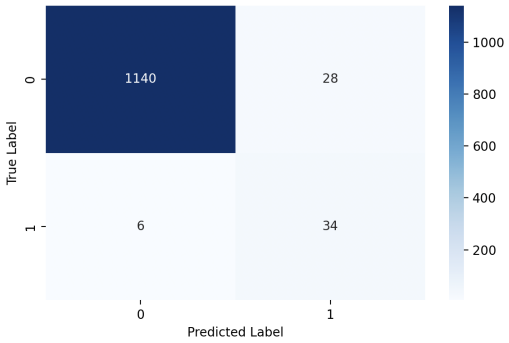



Fig. 5. Confusion matrix for Logistic Regression.

## IV. Results

### A. Model Evaluation

We analyzed the performance of three classifiers: Logistic Regression, SVM, and Random Forest across the dataset of 4,028 posts. Note that the dataset is heavily imbalanced, with the majority class size of 3,891 (96.7%) and minority class size of 134 (3.3%) (see Figure 4).

Confusion matrices are generated to visualize the distribution of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) for each classifier under the high class imbalance (see Figure 5, 6, 7).

From Figure 8, we observe that Logistic Regression results in an accuracy, precision, recall, and F1-score of 97.19%, 54.84%, 85%, and 66.67% respectively. SVM achieves 97.35%, 56.90%, 82.50%, and 67.35%, respectively. Random Forest achieves 96.94%, 52.46%, 80%, and 63.37%, respectively. SVM yields the highest accurary, precision, and F1-score, while Logistic Regression achieves the highest recall.

In addition to accuracy, precision, recall, and F1-score, we also evaluate performance through Area Under the Receiver Operating Characteristic Curve (ROC-AUC) and the Area Under the Precision–Recall Curve (PR-AUC) (see Figure {9 and Figure 10). ROC-AUC scores for Logistic Regression, SVM, and Random Forest are similarly high at 0.9712, 0.9747,

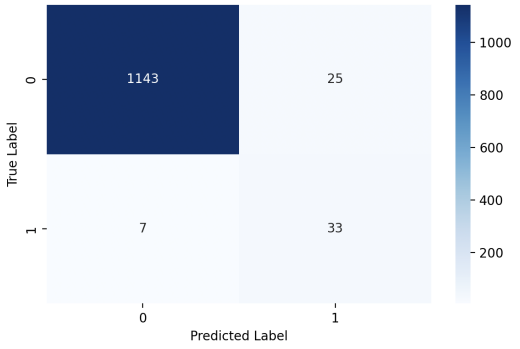



Fig. 6. Confusion matrix for SVM.

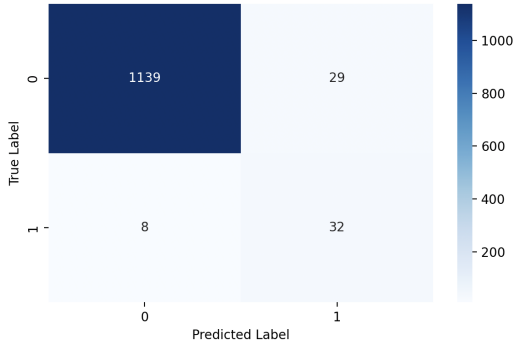



Fig. 7. Confusion matrix for Random Forest.

and 0.9736, respectively. We observe that SVM achieves the highest PR-AUC score (0.7884), followed by Logistic Regression (0.7544), and Random Forest (0.6764).

### B. Data and Prototype

The prototype frontend (see Figure 11) illustrates how software release data and Reddit discussions are integrated into a unified view. Each new software release or CVE entry is automatically linked with Reddit posts detected as update-related risks by the machine learning classifier. The matched posts are displayed within the corresponding update card and sorted in chronological order, ensuring that the most recent risks appear first. Each entry includes the Reddit title, subreddit source, timestamp, and model-assigned confidence score, allowing developers to inspect emerging reliability or compatibility issues immediately after release.

This integration demonstrates the connection between structured release metadata and unstructured community feedback, bridging the gap between software evolution and user-experienced risks. The interface supports paging and limit parameters to scale across large datasets while maintaining responsive filtering by component, release date, or update type (e.g., major, minor, patch).

For reproducibility and verification, a sample of positively labeled Reddit posts with their associated risk scores is publicly available[1].

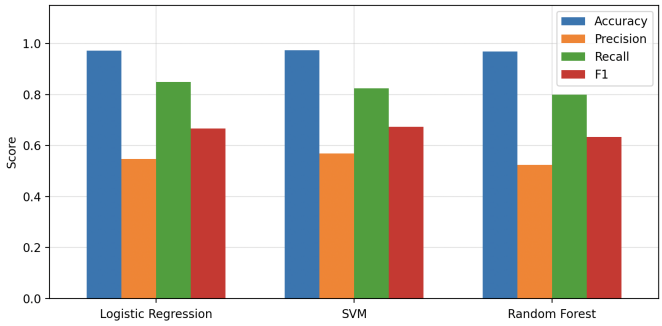



Fig. 8. Performance comparison between Logistic Regression, SVM, and Random Forest.

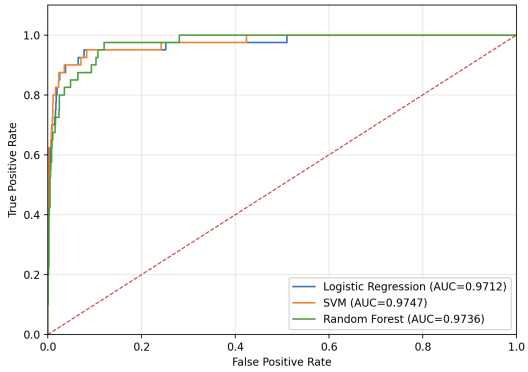



Fig. 9. Receiver Operating Characteristic (ROC) curves comparing Logistic Regression, SVM, and Random Forest.

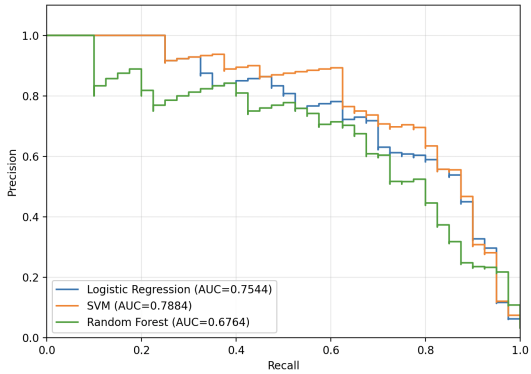



Fig. 10. Precision–Recall (PR) curves comparing Logistic Regression, SVM, and Random Forest.

## V. Discussion

### A. Machine Learning Discussion

*a) Labeling criteria.:* Our labeling criteria are intentionally selective, as mentioned in Section III-B. A post is labeled as True only if the user clearly describes an issue that occurred *after* performing an update. This strict guideline

[1]Positive-labeled Reddit posts on update-related risks are available at: https://www.kaggle.com/datasets/softwareupdatedata/reddit-posts-on-risks-with-software-updates

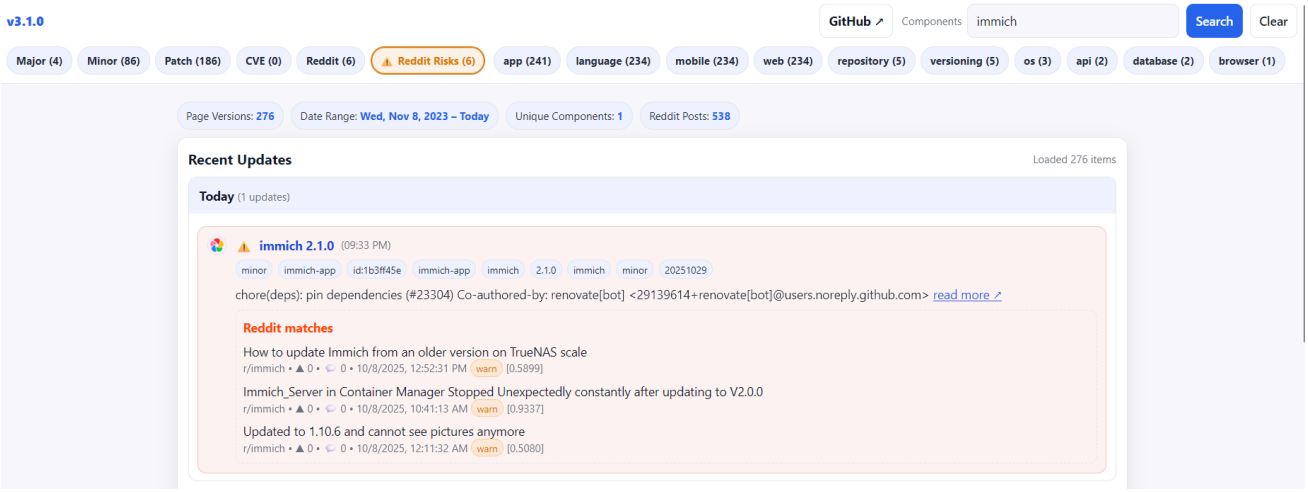



Fig. 11. Prototype dashboard linking software updates with Reddit risk posts, showing risk levels.

reduces ambiguity and improves consistency in the labeling process, but also results in relatively few positive examples. To address this imbalance, we applied random oversampling and assigned higher weights to the minority class during training. However, the limited number of minority samples affects some metrics in our evaluation.

*b) Evaluation criteria under class imbalance.:* As the dataset is heavily imbalanced, accuracy alone is not sufficient to compare the performance between three classifiers. Recall, F1-score and PR-AUC are more informative for the evaluation. Among the three classifiers, Logistic Regression achieves the highest recall of 85%, which minimizes false negatives at the cost of lower precision. SVM achieves the highest F1-score of 67.35%, which is the balance between precision and recall. SVM achieves the highest PR-AUC of 0.7884. This is 23.7x the baseline PR-AUC of 0.0333, which indicates that it strongly ranks true positives and keeps the best trade-off between precision and recall among the three classifiers. However, our models have a disadvantage in precision because of the limited number of positive labels. This means that a single misclassification can affect the precision noticeably. Given our goal of prioritizing early risk detection, high recall is more valuable than precision because missing a relevant post is costlier than flagging a non-relevant one.

*c) Decision boundary analysis.:* We also examined four posts to understand how our model distinguishes between update-related and not update-related posts and its decision boundary.

- **Min negative**: r/Python: *"[Release] py-github-analyzer - High-performance async GitHub repo analyzer for AI/ML workflows"* [15].
  Our model is highly confident that this post is not update-related and assigns its label False.
- **Max negative**: r/Chrome *"How to fix the disappearing of icons? And what is this problem exactly am I the only one?"* [16].
  This post is close to the decision boundary of 0.5, and our model predicts its label is False with lower confidence.
- **Min positive**: r/Windows *"Windows update somehow deletes a plane from GTA San Andreas after 20 years, but it's not Microsoft's fault"* [17].
  This post also lies near the decision boundary, and our model predicts its label is True with lower confidence.
- **Max positive**: r/Chrome *"Lost all tab groups after updating to version 135.0.7049.100 (Chrome Mobile)"* [18].
  Our model is highly confident this post clearly describes a technical issue after an update and assigns its label True.

## B. System Integration Discussion

From a system perspective, integrating a text-classification pipeline into an existing release- and component-tracking platform raises several operational constraints that are not visible from the ML metrics alone.

*a) Stable labeling timeline.:* The first constraint is temporal consistency of labels. Our 4,028 manually labeled Reddit posts were created at an earlier time and reflect the platform state then. If we label *live* Reddit objects directly in the production collection (e.g., `reddit`) and later re-run ingestion, the new pass can overwrite the human label or mix model-generated labels with human labels. To avoid this, labeling should operate on a frozen snapshot (time-bounded export) and write back to a separate labeling collection or field namespace (e.g., `metadata.labeling.v1.*`). This keeps historical labels reproducible and allows retraining and ablation studies without being affected by in-place updates.

*b) Baseline runtime environment for ML.:* The platform performs enrichment in intervals (every few hours). If the underlying Python runtime or library stack differs between intervals, the worker is forced to create a new virtual environment and re-install packages before every prediction run, which introduces avoidable latency and can desynchronize model versions. A practical requirement is to standardize on (i) a baseline Python version, (ii) an explicit requirements file, and (iii) a model-artifact

version tag that is stored alongside the prediction output. In our case, storing `ml.runtime.python_version`, `ml.runtime.requirements_hash`, and `ml.model.version` with each prediction makes it possible to trace which classifier produced which label during later audits.

*c) Safe batch prediction on large Reddit partitions.:* The data lake currently hosts about 85k Reddit posts. Even though the classifier is fast for a single post, a full-batch prediction over the entire collection can fail due to a single malformed record or due to temporary network or storage contention. Therefore, the system should always support a "small-sample dry run" mode (e.g., first 200–500 posts from the current window) before scaling to the full partition. Only if the small sample passes schema validation and produces confidence scores in the expected range should the job proceed to the full 85k batch. In addition, the prediction API should expose explicit `limit` and `page`/`offset` parameters (e.g., `/api/predict?limit=500&page=3`) so that long-running predictions can be executed in deterministic chunks, retried per page, and monitored independently. This paging model minimizes the risk of partial writes and inconsistent model coverage across days because a failed page can be re-issued without recomputing the entire batch.

*d) Schema separation for ML outputs.:* Reddit objects ingested from the source API should remain close to their original JSON structure (author, title, selftext, subreddit, timestamps). ML-derived fields such as "is update related," "risk score," or "component match" should be attached under separate sub-documents to avoid name collisions and to support multiple concurrent models. A concrete pattern is:

- `metadata.predicted.*` for predicted score and timestamp,
- `metadata.labeling.*` for human labels, and
- `ml.[model_name].*` for each classifier (e.g., `ml.update_risk.v1.label`

This allows us to add a second or third model (e.g., "security regression detector," "UI breakage detector") without rewriting existing posts and without losing the human-labeled ground truth.

*e) Frontend Integration and Filtering.:* Once the predictions are written back to the same data lake that serves the dashboard, the frontend must expose the ML signals as first-class filters. In our setting this means filtering by (i) risk score or confidence band, (ii) affected component or update path, and (iii) chronological order. Chronological filtering is important because update-related complaints are time-bound to a specific release date; component-based filtering is important because the same user complaint may match multiple components in the release database. Providing these three filters makes the ML output immediately actionable for operators who track new releases every day.

*f) End-to-End Data Quality Dimensions.:* At the system level, the integration pipeline supports data quality across seven dimensions that ensure the reliability of update-risk detection. *Timeliness* is achieved through near real-time ingestion and prediction of update-related risks. *Accuracy* is maintained by applying supervised machine learning classifiers trained on verified labels. *Variety* is supported through the integration of heterogeneous text sources drawn from Reddit posts. *Completeness* is addressed by linking Reddit discussions with CVE, GitHub, and release data. *Consistency* is ensured through uniform labeling, schema design, and namespace structure across all endpoints. *Validity* is improved by filtering out irrelevant or noisy records before enrichment. Finally, *Uniqueness* is maintained by consolidating related component-specific open text posts with existing CVE advisories and official release notes.

*g) (Practical implications.:* The proposed system extends beyond research evaluation to direct operational use within continuous deployment (CD) pipelines. Organizations such as operating system vendors or IoT providers can integrate the model into their deployment workflows to detect community-reported update issues within minutes of release, reducing mean time to awareness (MTTA) and improving patch management. By feeding user-derived risk signals back into post-deployment dashboards, the system aims to enable continuous monitoring of software reliability.

Overall, these system-level choices ensure that the model can be retrained on earlier labels, that predictions remain reproducible across enrichment intervals, and that multiple ML algorithms can coexist on the same Reddit object without mutual overwrite.

## VI. Limitations and Future Work

*a) Expand training and testing dataset.:* The current labeled dataset only contains 4,028 posts, which may not fully cover the diversity of real-world data and thus limit our results. Future work could expand the dataset through manual or automated labeling methods, such as active learning, to increase data diversity for better evaluation.

*b) Expand to other open text sources.:* Our work uses Reddit as the primary data source. Future work could extend it to other open-source communities, such as GitHub, X, app store reviews. Expanding the data could help identify update-related risks more reliably.

*c) Expand feature selection.:* Our work uses a combination of boolean flags and the TF-IDF features. Future study could expand feature representations using other word embedding methods such as Word2Vec.

*d) (Implement other models.:* Our study uses three classifier: Logistic Regression, Support Vector Machine, and Random Forest. Future work could implement other classifiers such as Naïve Bayes and K Nearest Neighbors, or explore ensemble methods such as Gradient Boosting and Adaboost. Evaluating different models can provide more information about the data and can improve the overall performance metrics.

*e) Risk-labeled data for vulnerability research.:* The ability to filter Reddit posts by software update risks enables the creation of a reusable ground truth for studying how risks

are communicated and how they relate to *n*-day vulnerabilities or subsequent patches. Future research can leverage this dataset to examine early signals of emerging risks and the dynamics of community response to software updates.

*f) Open library implementation.:* A planned next step is to develop a lightweight library in Python or Node.js that encapsulates the model and data integration logic, allowing others to install and integrate the risk-detection module directly into their own monitoring or update-tracking systems. Such a library would simplify adoption, support reproducible research, and enable wider application of the approach across different software ecosystems.

*g) Real user evaluation feedback.:* While the prototype demonstrates technical feasibility, future work will include user-centered evaluations with practitioners who manage software updates in real environments. Focus group studies could assess how effectively the system's risk scores, filters, and dashboards support decision-making. Feedback from developers, system administrators, and security engineers would provide valuable feedback on usability and the practical impact of early risk detection in their workflows.

## VII. Conclusion

Our research found that a small but operationally important share of Reddit posts on software updates, about 3.3% in our manually labeled sample of 4,028 posts, actually describe risks along the software update path. To find this minority class faster and in an automated manner, this paper introduced a realtime open-text pipeline that collects software-related discussions, normalizes them across sources, and applies supervised machine learning to flag update-related failures as they appear. Despite the heavy class imbalance, models such as SVM and Logistic Regression achieved high recall and strong PR-AUC, which is the right optimization target when early detection matters more than perfect precision.

Beyond model metrics, we integrated the classifier into an existing release and component tracking system, added paging and limit parameters to the prediction API, and separated ML output fields from the original Reddit JSON to support multiple concurrent models. The prototype dashboard showed that newly ingested releases or CVEs can be linked immediately with risk-ranked Reddit posts, giving maintainers a unified view of what users report right after an update. By releasing a verified sample of positively labeled posts, we also enable follow-on work on risk communication, alignment with *n*-day vulnerabilities and patches, and cross-platform analysis. Future work will extend the dataset to other open text sources and package the logic as a reusable Python or Node.js library so that other teams can adopt update-risk detection without rebuilding the entire pipeline.

## Acknowledgements

We thank the open data and research communities for continuously publishing software-related content across public platforms and for providing APIs that enable programmatic access and reproducible analysis. We thank platforms such as Kaggle and GitHub for enabling public sharing and version control of the datasets and software used in this work. Their openness and data-sharing efforts make it possible to study discussions of software updates across communities.

## References

[1] R. Wash, E. Rader, K. Vaniea, and M. Rizor, "Out of the loop: How automated software updates cause unintended security consequences," in *Proceedings of the 10th Symposium on Usable Privacy and Security (SOUPS 2014)*. USENIX Association, 2014, pp. 89–104.

[2] K. Vaniea and Y. Rashidi, "Tales of software updates: The process of updating software," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2016, pp. 3215–3226.

[3] Reddit, "OpenGL is broken since latest update 139.0.7258.67," 2025, reddit post, r/chrome, Aug. 2025. [Online]. Available: https://reddit.com/r/chrome/comments/1mj1cnc/opengl_is_broken_since_latest_update_1390725867/.

[4] ——, "Edge hangs and crashes after update from version 140.0.3485.94 to version 141.0.3537.57," 2025, reddit post, r/MicrosoftEdge, Oct. 2025. [Online]. Available: https://reddit.com/r/MicrosoftEdge/comments/1nwkc4k/edge_hangs_and_crash_after_update_from_version/.

[5] ——, "Black screen after Windows update," 2025, reddit post, r/Dell, Jun. 2025. [Online]. Available: https://reddit.com/r/Dell/comments/1l870hd/black_screen_after_windows_update/.

[6] ——, "GPT header corruption after upgrading to TrueNAS scale 25.10," 2025, reddit post, r/truenas, Oct. 2025. [Online]. Available: https://reddit.com/r/truenas/comments/1oimbe6/gpt_header_corruption_after_upgrading_to_truenas/.

[7] ——, "DB not starting after V2.0.0 upgrade," 2025, reddit post, r/immich, Oct. 2025. [Online]. Available: https://reddit.com/r/immich/comments/1nwdfgi/db_not_starting_after_v200_upgrade/.

[8] CyberSecureFox Editorial Team, "Nist overhauls nvd: What "not scheduled" cves mean for vulnerability management in 2026," Apr. 2026, online; accessed 2026-05-31. [Online]. Available: https://cybersecurefox.com/en/nist-nvd-reform-not-scheduled-cve-2026/

[9] A. Fiallos and R. Jimenes, "Multi-label classification of twitter users' interests using reddit data," in *Proceedings of the 2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2019, pp. 4120–4127.

[10] R. Shrestha, S. Sharma, P. Pokhrel, and J. Kim, "Cross-platform analysis of software vulnerability communication: A study of GitHub, Twitter, and Reddit," *Information and Software Technology*, vol. 121, p. 106276, 2020.

[11] A. Mockus and D. M. Weiss, "Predicting risk of software changes," *Bell Labs Technical Journal*, vol. 5, no. 2, pp. 169–180, 2000.

[12] M. Iqbal, T. Ahmad, S. Rahman, and L. Chen, "Mining software requirements from Reddit using machine learning techniques," *Empirical Software Engineering*, vol. 26, no. 5, p. 98, 2021.

[13] Y. Long, Z. Wang, and H. Yu, "Comparing TextCNN and TextRNN models for Reddit text classification," *Expert Systems with Applications*, vol. 241, p. 122838, 2024.

[14] B. Singh, S. M. Daftary, M. Rudra, S. P. Veed, S. Berhe, M. Maynard, and F. Khomh, "Enriching release notes with Reddit feedback: Understanding how user feedback can augment maintaining software updates," in *Competitive Tools, Techniques, and Methods (Chronicle of Computing, SERD24#11)*, P. F. Tiako, Ed. OkIP, 2024.

[15] Reddit, "[release] py-github-analyzer – high-performance async GitHub repo analyzer for AI/ML workflows," 2025, reddit post, r/Python, Oct. 2025. [Online]. Available: https://www.reddit.com/r/Python/comments/1nze1oo/release_pygithubanalyzer_highperformance_async/.

[16] ——, "How to fix the disappearing of icons? and what is this problem exactly am i the only one?" 2025, reddit post, r/chrome, Oct. 2025. [Online]. Available: https://reddit.com/r/chrome/comments/1nwalqy/how_to_fix_the_disappearing_of_icons_and_what_is/.

[17] ——, "Windows update somehow deletes a plane from GTA San Andreas after 20 years, but it is not Microsoft's fault," 2025, reddit post, r/Windows, Apr. 2025. [Online]. Available: https://www.reddit.com/r/windows/comments/1k6p0dn/windows_update_somehow_deletes_a_plane_from_gta/.

[18] ——, "Lost all tab groups after updating to version 135.0.7049.100 (Chrome mobile)," 2025, reddit post, r/chrome, Apr. 2025. [Online]. Available: https://www.reddit.com/r/chrome/comments/1k20y1b/lost_all_tab_groups_after_updating_to_version/.